AI ASSISTED CODING

ASSIGNMENT-10.1

Name:V.Sruthi

H.no:2303A51308

Batch:05

TASK DESCRIPTION #1 – SYNTAX AND LOGIC ERRORS

PROMPT:

```
"""def calc_average(marks):
    total = 0
    for m in marks:
        total += m
   average = total / len(marks)
   return avrage # Typo here
marks = [85, 90, 78, 92]
print("Average Score is ", calc_average(marks)"""
```

Identify and fix all syntax and logic errors in the given Python code, provide the corrected runnable version, and briefly explain the changes made.

```
10.1-1.py > ...
"""def calc_average(marks):
  total=0
  for m in marks:
    total+=m
  average=total/len(marks)
  return avrage # Typo here
marks=[85, 90, 78, 92]
print("Average Score is ", calc_average(marks)"""

#Identify and fix all syntax and logic errors in the given Python code, provide the corrected runnable version, and briefly explain the changes made."

def calc_average(marks):
    total = 0
    for m in marks:
        total += m
    average = total / len(marks)  # Fixed typo here
    return average  # Fixed typo here
marks = [85, 90, 78, 92]
print("Average Score is ", calc_average(marks))  # Added missing parenthesis here
```

OUTPUT:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   POSTMAN CONSOLE

PS C:\Users\SRUTHI\Desktop\ai assistant> & C:/Users/SRUTHI/anaconda3/python.exe "c:/Users/SRUTHI/Desktop/ai assistant/10.1-1.py"
Average Score is  86.25
PS C:\Users\SRUTHI\Desktop\ai assistant> & C:/Users/SRUTHI/anaconda3/python.exe "c:/Users/SRUTHI/Desktop/ai assistant/10.1-1.py"
PS C:\Users\SRUTHI\Desktop\ai assistant> & C:/Users/SRUTHI/anaconda3/python.exe "c:/Users/SRUTHI/Desktop/ai assistant/10.1-1.py"
Average Score is  86.25
PS C:\Users\SRUTHI\Desktop\ai assistant> & C:/Users/SRUTHI/anaconda3/python.exe "c:/Users/SRUTHI/Desktop/ai assistant/10.1-1.py"
Average Score is  86.25
PS C:\Users\SRUTHI\Desktop\ai assistant>
```

EXPLANATION:

- Indentation was missing inside the function.
- avrage was a typo — corrected to average.
- The print statement was missing a closing bracket).
- After fixing these errors, the code runs correctly and gives the average score.

TASK DESCRIPTION #2 – PEP 8 COMPLIANCE

PROMPT:

“”””def area_of_rect(L,B) : return L*B

print(area_of_rect(10,20))”””

Refactor the given Python code to follow PEP 8 style guidelines by correcting indentation, spacing, variable naming, and formatting. Provide the well-structured, readable, and PEP 8-compliant version of the code.

```python
"""def area_of_rect(L,B) : return L*B
print(area_of_rect(10,20))"""

#Refactor the given Python code to follow PEP 8 style guidelines by correcting indentation, spacing, variable naming, and formatting.
# Provide the well-structured, readable, and PEP 8-compliant version of the code.
def area_of_rectangle(length, breadth):
    return length * breadth
print(area_of_rectangle(10, 20))
```

OUTPUT:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE

PS C:\Users\SRUTHI\Desktop\ai assistant> & C:/Users/SRUTHI/anaconda3/python.exe "c:/Users/SRUTHI/Desktop/ai assistant/10.1-2.py"
PS C:\Users\SRUTHI\Desktop\ai assistant> & C:/Users/SRUTHI/anaconda3/python.exe "c:/Users/SRUTHI/Desktop/ai assistant/10.1-2.py"
200
PS C:\Users\SRUTHI\Desktop\ai assistant> & C:/Users/SRUTHI/anaconda3/python.exe "c:/Users/SRUTHI/Desktop/ai assistant/10.1-2.py"
200
PS C:\Users\SRUTHI\Desktop\ai assistant>
```

EXPLANATION:

- Variable names L and B were changed to length and breadth for better readability.
- Removed the one-line function definition and placed the return statement on a new indented line.
- Removed the extra space before the colon :
- Added proper spacing around operators (*).
- Added proper spacing after commas in function arguments.
- Now the code follows PEP 8 style guidelines and is clean and readable.

TASK DESCRIPTION #3 – READABILITY ENHANCEMENT

PROMPT:

def c(x,y):

return x*y/100

a=200

b=15

print(c(a,b))

Improve the readability of the given Python code without changing its logic. Rename variables and functions to be more descriptive, format the code properly, and add meaningful inline comments to make it clear and easy to understand.

```
10.1-3.py > ...
def c(x,y):
 return x*y/100
a=200
b=15
print(c(a,b))
#Improve the readability of the given Python code without changing its logic.
# Rename variables and functions to be more descriptive, format the code properly, and add meaningful inline comments to make it clear and easy to understand
def calculate_percentage(value, percentage):
    """Calculate the percentage of a given value."""
    return value * percentage / 100
# Define the value and percentage to be calculated
amount = 200
percentage_to_calculate = 15
# Calculate and print the result
result = calculate_percentage(amount, percentage_to_calculate)
print(result)
```

OUTPUT:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   POSTMAN CONSOLE

PS C:\Users\SRUTHI\Desktop\ai assistant> & C:/Users/SRUTHI/anaconda3/python.exe "c:/Users/SRUTHI/Desktop/ai assistant/10.1-3.py"
30.0
30.0
PS C:\Users\SRUTHI\Desktop\ai assistant> & C:/Users/SRUTHI/anaconda3/python.exe "c:/Users/SRUTHI/Desktop/ai assistant/10.1-3.py"
30.0
30.0
```

EXPLANATION:

- The function name c was unclear, so it was changed to calculate_percentage to show what it does.
- Variable names x and y were renamed to amount and percentage for better understanding.

- Variables a and b were changed to total_amount and discount_percentage to make their purpose clear.
- Comments were added to explain each part of the code.
- Proper spacing and formatting were used to improve readability.

- The logic of the program was not changed — only the clarity was improved.

TASK DESCRIPTION #4 – REFACTORING FOR MAINTAINABILITY

PROMPT:

students = ["Alice", "Bob", "Charlie"]

print("Welcome", students[0])

print("Welcome", students[1])

print("Welcome", students[2])

Refactor the given Python code to improve maintainability by removing repetitive statements and creating reusable functions. Rewrite the code in a modular way while keeping the same output.

```
10.1-4.py > ...
students = ["Alice", "Bob", "Charlie"]
print("Welcome", students[0])
print("Welcome", students[1])
print("Welcome", students[2])
#Refactor the given Python code to improve maintainability by removing repetitive statements and creating reusable functions.
# Rewrite the code in a modular way while keeping the same output.
def welcome_student(student):
    """Print a welcome message for the given student."""
    print("Welcome", student)
students = ["Alice", "Bob", "Charlie"]
for student in students:
    welcome_student(student)
```

OUTPUT:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE

Welcome Alice
Welcome Bob
Welcome Charlie
Welcome Alice
Welcome Bob
Welcome Charlie
PS C:\Users\SRUTHI\Desktop\ai assistant>
```

EXPLANATION:

- Created a reusable function welcome_student() to avoid repeating the print statement.
- Used a for loop to go through the list instead of accessing each index manually.

- This makes the code easier to maintain — if more students are added, no extra print statements are needed.
- The code is now cleaner, modular, and scalable.

TASK DESCRIPTION #5 – PERFORMANCE OPTIMIZATION

PROMPT:

nums = [i for i in range(1,1000000)]

squares = []

for n in nums:

    squares.append(n**2)

print(len(squares))

Optimize the given Python code to improve its performance without changing its output. Refactor it using more efficient approaches such as list comprehensions or other optimized techniques, and provide the improved version of the code.

```
lab-11.3.py   11.3-2.py   12.4-1.py   12.py   10.1-1.py   10.1-2.py   10.1-3.py   10.1-4.py   10.1-5.py   12.4-2.py   12.4-3.py
10.1-5.py > ...
# Find squares of numbers
nums=[i for i in range(1,1000000)]
squares=[]
for n in nums:
  squares.append(n**2)
print(len(squares))
#Optimize the given Python code to improve its performance without changing its output.
# Refactor it using more efficient approaches such as list comprehensions or other optimized techniques, and provide the improved version of the code.
# Find squares of numbers
nums = range(1, 1000000)  # Using range instead of list comprehension
squares = [n**2 for n in nums]  # Using list comprehension for better performance
print(len(squares))
```

OUTPUT:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   POSTMAN CONSOLE

999999
999999
PS C:\Users\SRUTHI\Desktop\ai assistant>
```

EXPLANATION:

- Replaced the manual for loop with a list comprehension, which is faster and cleaner.

- Used range() directly instead of creating a separate list (nums), saving memory.
- The generator version avoids storing all squares in memory, improving efficiency for large datasets.
- The optimized versions run faster and use memory more efficiently.

TASK DESCRIPTION #6 – COMPLEXITY REDUCTION

PROMPT:

```
def grade(score):
    if score >= 90:
        return "A"
    else:
        if score >= 80:
            return "B"
        else:
            if score >= 70:
                return "C"
            else:
                if score >= 60:
                    return "D"
                else:
                    return "F"
```

Simplify the given Python function to reduce complexity while keeping the same logic and output. Refactor the nested conditional statements into a cleaner structure using elif statements or an appropriate alternative such as dictionary mapping

```python
10.1-6.py > ...
def grade(score):
  if score >= 90:
     return "A"
  else:
    if score >= 80:
       return "B"
    else:
      if score >= 70:
         return "C"
      else:
        if score >= 60:
           return "D"
        else:
           return "F"
print(grade(85))
#Simplify the given Python function to reduce complexity while keeping the same logic and output.
# Refactor the nested conditional statements into a cleaner structure using elif statements or an appropriate alternative such as dictionary mapping.
def grade(score):
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    elif score >= 60:
        return "D"
    else:
        return "F"
print(grade(85))
```

OUTPUT:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE

B
B
PS C:\Users\SRUTHI\Desktop\ai assistant> & C:/Users/SRUTHI/anaconda3/python.exe "c:/Users/SRUTHI/Desktop/ai assistant/10.1-6.py"
B
B
PS C:\Users\SRUTHI\Desktop\ai assistant>
```

EXPLANATION:

- In the original function, multiple if statements were placed inside else blocks. This created deep nesting, making the code harder to read and understand. Even though the logic was correct, the structure was unnecessarily complex.
- In the improved version, the nested if-else statements were replaced with elif. The elif statement allows checking multiple conditions in a straight, clean sequence. This removes extra indentation and makes the code easier to follow.